SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Functions and Object-Oriented Programming

CO2: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Student Name	Nirmal Kumar T	**Register Number**	1972260020
Department	B.Tech CSE-AI	**Academic Year**	2026–2030
Section		**Semester**	2
Faculty Name	Amirtha S	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	12 November 2026	**Date of Evaluation**	—
GitHub Repository	DSA01-1972260020-Assignment2	**Compiler**	g++ 13.3.0, -std=c++17, -Wall -Wextra
Repository Link	https://github.com/Nirmal637/DSA01-1972260020-Assignment2		

MARK SHEET (Rubric-based Evaluation)

Q	Task Title	Max Marks	Marks Awarded	Remarks
1	Call by Value, by Reference and by Pointer	6		
2	Return by Reference and Its Dangers	4		
3	Default Arguments and Function Overloading	6		
4	Specifying a Class with an Array Inside It	6		
5	Static Members and Arrays of Objects	7		
6	Friend Functions and Inline Functions	8		
7	Diagnose, Redesign and Validate	13		
	TOTAL	50		Percentage:

Note: Marks awarded and remarks are intentionally left blank for faculty evaluation.

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual **Duration:** 1 Week **Total Marks:** 50

Submission Date: 12 November 2026 **Bloom's Level:** K5 – Evaluate (all questions)

Assignment Title: Functions and Object-Oriented Programming – Design, Implement and Validate C++ Solutions

Mandatory Execution and GitHub Submission

- Execute every question individually, including all "Complete the Code" programs.
- Maintain one folder for each question (Q1 through Q7), containing source files, any input files, and genuine output screenshots.
- Include a README.md with student name, register number, compiler/version, exact compile command, run steps, and a short result note for every question.
- Commit progressively, with at least one commit per question. Do not upload everything as one final commit.
- Share the repository with faculty (public, or private with faculty added as collaborator). A working repository link is mandatory.

GitHub Repository Link: https://github.com/Nirmal637/DSA01-1972260020-Assignment2

How to Answer the Additional Questions

Complete the Code: Replace each TODO with working code without changing unrelated supplied code; compile, run and compare against the expected output.

Predict the Output: Record the exact prediction before running the program. Then compare it with the actual output and explain every difference using the relevant C++ rules.

Alternative Concept and Complexity Analysis: Explain the alternative design, provide code or pseudocode, and compare time and space using Big-O. Record measurements only after running the programs yourself.

Evidence integrity: This report provides predicted outputs and compile-ready programs. No compiler screenshots or machine benchmark results are represented as your personal execution evidence. Capture your own terminal screenshots and fill in the evidence fields after running the programs with your stated compiler.

Q1 – CALL BY VALUE, BY REFERENCE AND BY POINTER (6 MARKS)

Problem context – Hostel Fee Concession Updater

The hostel office applies merit (10%), sports (15%) and single-parent (25%) concessions. The central issue is whether a function receives a copy of a fee or access to the caller's fee object.

1. Parameter behaviour and comparison

Passing method	Parameter represents	Changes caller's fee?	When appropriate
By value: long long fee	A separate local copy of the number	No	Small scalar inputs when the function should not mutate the caller
By reference: long long& fee	An alias for the caller's variable	Yes	A required in-place update; no copy of the scalar
By pointer: long long* fee	An address; *fee accesses the original object	Yes, when non-null	Optional/nullable target or APIs that explicitly use pointer semantics
By const reference: const Student&	Read-only alias to an existing object	No	Large objects that must be inspected without copying or mutation

A by-value parameter is initialised from the argument, so arithmetic changes only the local parameter. A reference parameter is another name for the caller's object. A pointer parameter stores an address; dereferencing it changes the object at that address. A pointer should be checked for nullptr before dereferencing.

2. Design decision

For one simple concession, returning the revised fee is often easiest to reason about because the caller explicitly assigns the result. For a workflow that must update a Student object in place, a reference-based member operation can be appropriate. The implementation below demonstrates all three required passing styles. It uses integer fee units and integer percentage arithmetic; the fee values are whole rupees in this demonstration.

3. Predicted output for the supplied program

```
1 12 13
1 100
16 13 6
```

Explanation: change(a,b,&c;) increments only the local copy a, but updates b through its reference and c through its pointer. Therefore a=1, b=12, c=13. bigger(a,c) returns a reference to the larger object; c is larger, so the assignment changes c to 100. show(1) uses y=5 and z=10; show(1,2) uses z=10; show(1,2,3) uses no defaults.

4. Complete implementation – three passing styles

```
#include <iostream>

void applyByValue(long long fee, int percent) {
fee -= fee * percent / 100;
}

void applyByReference(long long& fee, int percent) {
fee -= fee * percent / 100;
}

void applyByPointer(long long* fee, int percent) {
if (fee != nullptr)
*fee -= *fee * percent / 100;
}

int main() {
long long merit = 45000;
long long sports = 45000;
long long singleParent = 45000;

applyByValue(merit, 10);
applyByPointer(&sports, 15);
applyByReference(singleParent, 25);

std::cout << "Value: " << merit << '\n';
std::cout << "Pointer: " << sports << '\n';
std::cout << "Reference: " << singleParent << '\n';
}
```

Expected output: Value: 45000; Pointer: 38250; Reference: 33750. The by-value call leaves merit unchanged because only its local parameter is modified. The pointer and reference calls update the original variables.

5. Alternative: return the new fee

```
long long applyConcession(long long fee, int percent) {
return fee - fee * percent / 100;
}

// Call:
hostelFee = applyConcession(hostelFee, 10);
```

All three scalar versions take O(1) time. Passing a long long by value copies one scalar (constant space); reference, pointer and const-reference passing store an address/reference-sized handle (also O(1)). Passing a large Student object by value copies all its fields and may copy owned resources; passing by reference or const reference avoids that object copy. A const reference is preferred for a large read-only object.

Execution evidence to add: Run the prediction program and the concession implementation. Insert a screenshot showing the compile command, no compiler diagnostics (if none occur), and the exact terminal output. Do not treat the predicted output above as an observed screenshot.

Q2 – RETURN BY REFERENCE AND ITS DANGERS (4 MARKS)

Problem context – Examination Marks Register

The marks array stores a fixed set of student marks. Returning int& to the highest array element allows the caller to modify that element directly. The reference must refer to an object that remains alive after the function returns.

1. Working functions and demonstration

```
#include <iostream>

int& highestMark(int (&marks)[5]) {
int index = 0;
for (int i = 1; i < 5; ++i)
if (marks[i] > marks[index]) index = i;
return marks[index];
}

const int& highestMarkReadOnly(const int (&marks)[5]) {
int index = 0;
for (int i = 1; i < 5; ++i)
if (marks[i] > marks[index]) index = i;
return marks[index];
}

int highestMarkCopy(const int (&marks)[5]) {
int index = 0;
for (int i = 1; i < 5; ++i)
if (marks[i] > marks[index]) index = i;
return marks[index];
}

int main() {
int marks[5] = {78, 91, 84, 88, 95};
highestMark(marks) += 2;
std::cout << "Highest after moderation: "
<< highestMark(marks) << '\n';
std::cout << "Read-only highest: "
<< highestMarkReadOnly(marks) << '\n';
std::cout << "Copy of highest: "
<< highestMarkCopy(marks) << '\n';
}
```

Predicted output:

```
Highest after moderation: 97
Read-only highest: 97
Copy of highest: 97
```

highestMark(marks) returns an lvalue reference to marks[4], so +=2 modifies the original array element from 95 to 97. The const-reference version prevents modification through the returned reference. The copy version returns a separate integer; changing that returned value would not modify the array.

2. Dangling reference – unsafe example

```
int& badReference() {
int local = 42;
return local; // WRONG: local lifetime ends on return
}

// Do not call or dereference the returned reference.
```

A compiler such as g++ normally diagnoses returning a reference to a local variable with a warning such as “reference to local variable returned” or “returning reference to temporary”. Once the function exits, local no longer exists. The returned reference dangles; using it is undefined behaviour. This example is intentionally not called in the working program.

3. Reference, const reference and copy

Return type	Caller can modify source?	Safety / use
int&	Yes	Efficient writable alias; source object must outlive the reference
const int&	No, not through this reference	Efficient read-only alias; source lifetime still matters
int	No; receives a value copy	Independent result; safest when caller needs only a value

4. Changed condition – std::vector growth

When vector capacity is exceeded, reallocation moves its elements to a new allocation and invalidates all references, pointers and iterators to its old elements. A reference can remain valid after push_back only when no reallocation occurs; do not rely on that unless capacity is controlled. Returning an index is often safer across vector reallocation because the index can be applied to the vector again after growth. Recompute the index if insertions/erasures change element positions.

5. Alternative: index or pointer

```
#include <vector>

int highestIndex(const std::vector<int>& marks) {
if (marks.empty()) return -1;
int index = 0;
for (int i = 1; i < static_cast<int>(marks.size()); ++i)
if (marks[i] > marks[index]) index = i;
return index;
}

// Option A: return an index (reusable after vector reallocation)
int index = highestIndex(marks);
if (index >= 0) marks[index] += 2;

// Option B: return/use a pointer instead (invalidated by reallocation)
int* p = marks.empty() ? nullptr : &marks[highestIndex(marks)];
if (p != nullptr) *p += 2;
```

Finding the maximum is O(n) time and O(1) extra space whether the function returns a reference, index or pointer. An index survives vector reallocation as a numeric value, but must still be checked against the new size and may identify a different logical element after structural edits. A pointer/reference is directly tied to the old storage and is invalidated by reallocation. For a fixed C++ array, a reference is convenient; for a growable vector, returning an index is generally more robust.

Evidence to capture: A run showing 97 after moderation, plus a separate compile-only screenshot for the dangling-reference example showing the compiler diagnostic. Never execute or dereference the dangling reference.

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING (6 MARKS)

Problem context – Courier Charge Calculator

The original design declares three overloads that can all accept one argument because default values make the second and third overloads callable with fewer arguments. Therefore charge(2) has multiple viable candidates and the call is ambiguous. Default arguments do not create a distinct overload signature.

1. Redesigned overload set

Use a single overload for each distinct input type: int means grams, double means kilograms, and Parcel represents a structured parcel. Each overload has its own optional express and insured arguments where appropriate. The design has no duplicate one-argument double overload.

```
#include <iomanip>
#include <iostream>

struct Parcel {
double kilograms;
bool express = false;
bool insured = false;
};

double calculate(double kilograms, bool express, bool insured) {
double total = 50.0 + 30.0 * kilograms;
if (express) total += 50.0;
if (insured) total += total * 0.02;
return total;
}

double charge(int grams, bool express = false,
bool insured = false) {
std::cout << "[grams overload] ";
return calculate(static_cast<double>(grams) / 1000.0,
express, insured);
}

double charge(double kilograms, bool express = false,
bool insured = false) {
std::cout << "[kilograms overload] ";
return calculate(kilograms, express, insured);
}

double charge(const Parcel& parcel) {
std::cout << "[Parcel overload] ";
return calculate(parcel.kilograms,
parcel.express, parcel.insured);
}

int main() {
std::cout << std::fixed << std::setprecision(2);
std::cout << "charge(2): Rs " << charge(2) << '\n';
std::cout << "charge(2.0f): Rs " << charge(2.0f) << '\n';
std::cout << "charge(2.0): Rs " << charge(2.0) << '\n';
std::cout << "charge(500, true): Rs "
<< charge(500, true) << '\n';
std::cout << "charge(2.5, true, true): Rs "
<< charge(2.5, true, true) << '\n';
const Parcel p{1.5, false, true};
std::cout << "charge(Parcel): Rs " << charge(p) << '\n';
}
```

Pricing rule used for this illustrative program: base Rs 50 + Rs 30 per kg; express adds Rs 50; insurance adds 2% of the subtotal. This pricing formula is an explicit design assumption because the question paper does not prescribe courier rates.

2. Six overload predictions

Call	Selected overload	Reason
charge(2)	charge(int,...)	Exact match to int; no conversion needed
charge(2.0f)	charge(double,...)	float-to-double is a promotion; int would require a conversion
charge(2.0)	charge(double,...)	Exact match to double
charge(500, true)	charge(int,...)	Exact first-argument match; default insured=false
charge(2.5, true, true)	charge(double,...)	Exact match to double
charge(Parcel{...})	charge(const Parcel&)	Exact Parcel argument binding

3. Overload resolution and default argument rules

The compiler first forms the viable candidate set, then ranks implicit conversion sequences. An exact match is preferred to a promotion, and a promotion is preferred to a general conversion. If no unique best candidate exists, the call is ambiguous. Default arguments are supplied only after a function has been selected; they do not improve the conversion rank. Defaults must be supplied from the right because a caller cannot omit a middle argument while supplying a later positional argument.

4. Alternative: one Parcel structure

```
struct ParcelOptions {
double kilograms;
bool express = false;
bool insured = false;
};

// One function:
// double charge(const ParcelOptions& parcel);
```

A single ParcelOptions interface avoids overload ambiguity and scales well when options are added (for example fragile handling or signature confirmation). Overload resolution occurs at compile time and has no separate run-time search cost. Passing a small Parcel by const reference avoids copying it; passing by value copies the structure. Both designs compute the charge in O(1) time. Overloads give concise type-specific entry points; a parcel structure groups related options and is easier to extend.

Expected selection output: the six calls print the overload labels shown in the table. The numeric results from the stated illustrative tariff are 50.06, 110.00, 110.00, 115.00, 178.50 and 96.90 respectively. These are predicted program outputs, not screenshot evidence.

Evidence to capture: Compile and run the six-call program; capture the terminal output and a separate compile test demonstrating the original ambiguous overload set.

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT (6 MARKS)

Problem context – Savings Account Mini-Statement

Each SavingsAccount keeps its account number, balance, a private array of the ten most recent transactions and the number of valid entries. The class exposes deposit(), withdraw() and miniStatement(); internal storage remains private so callers cannot corrupt the account history or balance.

1. Complete class design using a circular index

```
#include <iomanip>
#include <iostream>
#include <string>

struct Transaction {
std::string type;
long long amountPaise;
};

class SavingsAccount {
private:
std::string accountNumber_;
long long balancePaise_;
Transaction transactions_[10];
int count_;
int next_;

void record(const std::string& type, long long amountPaise);

public:
SavingsAccount(const std::string& accountNumber,
long long openingBalancePaise)
: accountNumber_(accountNumber),
balancePaise_(openingBalancePaise),
count_(0), next_(0) {}

void deposit(long long amountPaise);
bool withdraw(long long amountPaise);
void miniStatement() const;
long long balance() const { return balancePaise_; }
};

void SavingsAccount::record(const std::string& type,
long long amountPaise) {
transactions_[next_] = {type, amountPaise};
next_ = (next_ + 1) % 10;
if (count_ < 10) ++count_;
}

void SavingsAccount::deposit(long long amountPaise) {
if (amountPaise <= 0) return;
balancePaise_ += amountPaise;
record("Deposit", amountPaise);
}

bool SavingsAccount::withdraw(long long amountPaise) {
if (amountPaise <= 0 || amountPaise > balancePaise_)
return false;
balancePaise_ -= amountPaise;
record("Withdraw", amountPaise);
return true;
}

void SavingsAccount::miniStatement() const {
std::cout << "Account: " << accountNumber_
<< "\nLast " << count_ << " transactions:\n";
const int oldest = (next_ - count_ + 10) % 10;
for (int i = 0; i < count_; ++i) {
const Transaction& t =
transactions_[(oldest + i) % 10];
std::cout << t.type << " Rs "
<< t.amountPaise / 100 << '.'
<< std::setw(2) << std::setfill('0')
<< t.amountPaise % 100
<< std::setfill(' ') << '\n';
}
std::cout << "Balance Rs " << balancePaise_ / 100
<< '.' << std::setw(2) << std::setfill('0')
<< balancePaise_ % 100 << std::setfill(' ')
<< '\n';
}

int main() {
SavingsAccount account("SB1001", 100000);
account.deposit(25000);
account.withdraw(10000);
for (int i = 1; i <= 10; ++i)
account.deposit(i * 100);
account.miniStatement();
}
```

2. Why the array is private

Private storage preserves the class invariant: only successful deposits and withdrawals should be recorded, and the balance must change consistently with the transaction log. If callers could edit the array or count directly, they could create an impossible statement or bypass the account operations. Access specifiers therefore protect the state and expose only the operations that are part of the account interface.

3. Eleventh transaction: shifting versus circular indexing

Design	Insertion when full	Time per insertion	Extra storage
Shift array	Move entries 1..9 to 0..8, then store newest at 9	O(k), k=10 here; O(10) for this fixed capacity	Fixed 10 entries
Circular index	Overwrite the oldest slot and advance next=(next+1)%10	O(1)	Fixed 10 entries plus indices

For a last-50 requirement, the circular-buffer design changes the capacity constant and the storage array size; the indexing logic remains the same. It avoids shifting on every insertion and therefore remains the better fit for frequent updates. A std::deque with push_back/pop_front also offers O(1) end operations in typical use, but the assignment asks for understanding the design; the fixed circular array demonstrates the indexing explicitly.

4. Predicted output for the demonstration

The first deposit and withdrawal are overwritten after ten later deposits. The statement therefore contains Deposit Rs 1.00 through Deposit Rs 10.00, in chronological order, and the balance is Rs 1,205.00.

```
Account: SB1001
Last 10 transactions:
Deposit Rs 1.00
Deposit Rs 2.00
Deposit Rs 3.00
Deposit Rs 4.00
Deposit Rs 5.00
Deposit Rs 6.00
Deposit Rs 7.00
Deposit Rs 8.00
Deposit Rs 9.00
Deposit Rs 10.00
Balance Rs 1205.00
```

Complexity: deposit/withdraw plus ring insertion is O(1); printing the mini-statement is O(k), where k is at most 10 (or 50 after the capacity change). Storage is O(k) per account. The shift-based design has O(k) insertion when full, while the circular design has O(1) insertion.

Evidence to capture: Run at least 12 transactions and capture the final mini-statement showing only the newest ten, plus one rejected withdrawal test.

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS (7 MARKS)

Problem context – College Bus Seat Tickets

The issued-seat count and seat capacity belong to the bus-ticketing class as a whole, not to an individual ticket. Each ticket has its own passenger name and assigned seat number. Therefore the shared count is static; seat and name are non-static object data.

1. Static-member design and completed TODOs

```
#include <iostream>
#include <string>

class BusTicket {
static int issued;
static const int CAPACITY = 3;
int seat;
std::string name;

public:
explicit BusTicket(const std::string& n)
: seat(0), name(n) {
if (issued < CAPACITY)
seat = ++issued;
}

static int count() {
return issued;
}

void print() const {
std::cout << name << ": "
<< (seat ? "Seat " + std::to_string(seat)
: std::string("Waiting"))
<< '\n';
}
};

int BusTicket::issued = 0;

int main() {
BusTicket trip[4] = {
BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")
};

for (const BusTicket& ticket : trip)
ticket.print();

std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

2. Why static is needed

- **static int issued:** one shared counter for all BusTicket objects.
- **static const int CAPACITY:** one class-wide capacity value.
- **int seat and std::string name:** separate values stored in each ticket.
- **static count():** callable as BusTicket::count() without an object.

A static member function has no this pointer because it is not called for one particular object. It can directly access static members, but it cannot directly access a non-static seat or name: those members require a particular object. A static data member is declared in the class and, in this C++17 example, the non-inline static int is defined once outside the class. The array creates four objects in order; each constructor attempts to issue the next seat.

3. Predicted output for the Token program

```
101 103 104 4
```

Token::next is initialised to 100 before main. The three elements of counter are constructed first, receiving IDs 101, 102 and 103. The extra object is then constructed and receives 104. Four constructors ran; issued() returns next-100, which is 4.

4. Predicted output for the BusTicket array

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5. Tickets created inside a function called twice

```
void createBatch(const std::string& first,
const std::string& second) {
BusTicket batch[2] = {
BusTicket(first), BusTicket(second)
};
for (const BusTicket& ticket : batch)
ticket.print();
}

int main() {
createBatch("Asha", "Bala");
createBatch("Chitra", "Dev");
std::cout << "Issued: " << BusTicket::count() << '\n';
}

// Uses the BusTicket class defined above.
```

The static counter survives the end of a function and the destruction of its local ticket array. It is not reset when createBatch() returns. With a capacity of three, the first batch may consume seats 1 and 2; the second call can issue seat 3 and place the next ticket on the waiting list. Once capacity is reached, later objects remain waiting unless the design explicitly models cancellations or a new bus.

6. Alternative: global counter or TicketOffice object

A global integer can count tickets in O(1) time and uses O(1) storage, but it is not encapsulated and can be modified by unrelated code. A static class member also uses one counter and O(1) time, while keeping ownership inside BusTicket. A TicketOffice object passed to each BusTicket also uses one counter and O(1) time, but it supports multiple independent buses because each office owns its own capacity and issued count. For a system with several buses, TicketOffice is the stronger design; one static counter is suitable only when all tickets share one global bus capacity.

Evidence to capture: Run the Token prediction and the BusTicket array program. Capture the four seat lines and issued count. Add a separate two-call function test to show that static state persists.

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS (8 MARKS)

Problem context – Train Journey Planner

Time stores hours and minutes privately. minutesBetween(a,b) treats both arguments equally and computes an absolute difference; earlier(a,b) compares the two times. Friend functions are non-members granted access to private members by the class.

1. Completed friend functions

```
#include <cstdlib>
#include <iostream>

class Time {
int h, m;

public:
Time(int hh, int mm) : h(hh), m(mm) {}

friend int minutesBetween(const Time& a, const Time& b);
friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b) {
const int first = a.h * 60 + a.m;
const int second = b.h * 60 + b.m;
return std::abs(first - second);
}

bool earlier(const Time& a, const Time& b) {
return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main() {
Time departure(9, 45), arrival(13, 10);
std::cout << minutesBetween(departure, arrival) << ' '
<< std::boolalpha
<< earlier(arrival, departure) << '\n';

Time beforeNoon(11, 55), afterNoon(12, 5);
std::cout << minutesBetween(beforeNoon, afterNoon) << ' '
<< earlier(beforeNoon, afterNoon) << '\n';

std::cout << minutesBetween(beforeNoon, beforeNoon) << ' '
<< earlier(beforeNoon, beforeNoon) << '\n';
}
```

2. Predicted output and hand check

```
205 false
10 true
0 false
```

09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight; the absolute difference is 205. 11:55 is 715 minutes and 12:05 is 725 minutes, so the difference is 10 and the first time is earlier. A time compared with itself has difference 0 and is not strictly earlier.

3. Friend function versus member function

Design	Call form	Access	Encapsulation
Friend non-member	minutesBetween(a,b)	Direct access to private fields because class grants friendship	Keeps the operation symmetric; friendship is a deliberate privilege
Member function	a.minutesTo(b)	Direct access to the current object and, if allowed, other Time data	Natural when operation conceptually belongs to the left-hand object
Public getters	minutes(a) / minutes(b)	Only public interface is used	No friendship, but exposes accessor methods

A friend function is called like an ordinary function and has no implicit this pointer. It can access private members of the class that declares it as a friend. A member function is called through an object and has a this pointer. Making h and m public would weaken encapsulation because every caller could create or modify invalid times such as hour 29 or minute 90; a constructor and validation would then be harder to enforce.

4. Inline functions and the growth investigation

A function defined inside a class is implicitly inline. The inline specifier is primarily an ODR/linkage permission for identical definitions in multiple translation units; it is not a command that forces the compiler to substitute the function body at every call. The optimiser decides whether inlining is profitable. Small operations may be inlined, but measurement is needed before claiming a speed improvement.

For the requested 10,000,000-call investigation, compare two functions with identical work: one marked inline and one ordinary out-of-line definition. Run the same input sequence and accumulate a checksum so the work remains observable. Build at -O0

and -O2, record compiler version and elapsed times, and repeat runs to reduce noise. An optimiser may inline either function at -O2, so the keyword does not guarantee a faster result.

```
#include <chrono>
#include <iostream>

struct ClockTime { int h, m; };

inline bool earlierInline(const ClockTime& a, const ClockTime& b) {
return a.h * 60 + a.m < b.h * 60 + b.m;
}

bool earlierNormal(const ClockTime& a, const ClockTime& b) {
return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main() {
constexpr long long N = 10000000;
long long inlineSum = 0, normalSum = 0;

auto startInline = std::chrono::steady_clock::now();
for (long long i = 0; i < N; ++i) {
ClockTime a{static_cast<int>(i % 24),
static_cast<int>((i * 7) % 60)};
ClockTime b{static_cast<int>((i * 13 + 1) % 24),
static_cast<int>((i * 17 + 2) % 60)};
inlineSum += earlierInline(a, b);
}
auto endInline = std::chrono::steady_clock::now();

auto startNormal = std::chrono::steady_clock::now();
for (long long i = 0; i < N; ++i) {
ClockTime a{static_cast<int>(i % 24),
static_cast<int>((i * 7) % 60)};
ClockTime b{static_cast<int>((i * 13 + 1) % 24),
static_cast<int>((i * 17 + 2) % 60)};
normalSum += earlierNormal(a, b);
}
auto endNormal = std::chrono::steady_clock::now();

volatile long long keep = inlineSum + normalSum;
const auto inlineMs =
std::chrono::duration<double, std::milli>(
endInline - startInline).count();
const auto normalMs =
std::chrono::duration<double, std::milli>(
endNormal - startNormal).count();

std::cout << "Checksums: " << inlineSum << ' '
<< normalSum << '\n';
std::cout << "Inline ms: " << inlineMs
<< "\nNormal ms: " << normalMs << '\n';
(void)keep;
}
```

```
g++ -std=c++17 -O0 -Wall -Wextra bench.cpp -o bench_O0
g++ -std=c++17 -O2 -Wall -Wextra bench.cpp -o bench_O2
./bench_O0
./bench_O2
```

At -O0, a non-inlined call may have measurable call overhead. At -O2, the optimiser may inline either version and the timings may converge. The result depends on compiler, processor, optimisation and benchmark design; no timing is asserted in this report.

5. Deployment decision and complexity

Keep the friend functions if the two-object operation is naturally symmetric and only these narrowly scoped operations need private access. If the class grows many operations, prefer member functions or public read-only getters to reduce friendship. Each time comparison performs a constant number of arithmetic operations: O(1) time and O(1) extra space in all three designs. Inline status changes possible constant factors, not the Big-O complexity.

Evidence to capture: Capture the expected three output lines and the four benchmark runs (-O0/-O2, inline/non-inline). Include your actual elapsed times and do not copy sample numbers from another machine.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected program: Q4 SavingsAccount transaction history

A. Diagnose the limitation (2 marks)

The simple shift-array version keeps the newest ten transactions by moving nine stored entries whenever a new transaction arrives after the array is full. Its insertion work is O(k) for k retained entries. The limitation is not the fixed array itself; it is the repeated movement of existing entries on every insertion. For a history size of 10 this cost is small, but it grows linearly if the history is expanded to 50, 500 or more entries.

Evidence requirement: Run the original shift implementation for the same transaction counts used in the redesigned version. Record the elapsed time and the final-history checksum. This report does not claim a user-machine baseline measurement.

B. Define the design objective (2 marks)

Objective: retain exactly the newest ten entries in chronological order while reducing the full-buffer insertion operation from O(k) element moves to O(1) writes. Correctness must be unchanged: after N insertions, both implementations must report the same ten newest values and the same checksum.

C. Design the revised solution (4 marks)

Replace shifting with a circular array and two indices: count records how many slots are valid; next identifies the slot to overwrite. On each insertion, write once at next, advance next modulo capacity, and increase count until full. To print in chronological order, compute oldest=(next+capacity-count)%capacity and traverse count slots modulo capacity. The design adds only two integer indices; the array capacity is unchanged.

Pseudocode

```
ADD(entry)
data[next] <- entry
next <- (next + 1) MOD CAPACITY
IF count < CAPACITY THEN
count <- count + 1
END IF

PRINT_HISTORY
oldest <- (next + CAPACITY - count) MOD CAPACITY
FOR i <- 0 TO count - 1
PRINT data[(oldest + i) MOD CAPACITY]
END FOR
```

Design sketch: SavingsAccount → Transaction[10], count, next. deposit()/withdraw() → record() → circular overwrite. miniStatement() → chronological traversal from oldest slot.

D. Predict before implementing (1 mark)

For N=2,000,000 sequential values 1 through N, both versions should retain N-9 through N. Their checksums should be identical: (1,999,991 + 2,000,000) × 10 / 2 = 19,999,955. The circular version should perform a constant number of operations per insertion; its elapsed time is predicted to grow approximately linearly with N, while the shift version performs up to nine assignments per full-buffer insertion. This is a prediction, not a measured result.

E. Implement and validate the redesign (2 marks)

```
#include <array>
#include <chrono>
#include <cstddef>
#include <iostream>

struct Entry { long long value; };

class ShiftLog {
static constexpr std::size_t K = 10;
std::array<Entry, K> data_{};
std::size_t count_ = 0;
public:
void add(Entry e) {
if (count_ < K) data_[count_++] = e;
else {
for (std::size_t i = 1; i < K; ++i)
data_[i - 1] = data_[i];
data_[K - 1] = e;
}
}
long long checksum() const {
long long sum = 0;
for (std::size_t i = 0; i < count_; ++i)
sum += data_[i].value;
return sum;
}
};

class CircularLog {
static constexpr std::size_t K = 10;
std::array<Entry, K> data_{};
std::size_t count_ = 0;
std::size_t next_ = 0;
public:
void add(Entry e) {
data_[next_] = e;
next_ = (next_ + 1) % K;
if (count_ < K) ++count_;
}
long long checksum() const {
long long sum = 0;
const std::size_t oldest = (next_ + K - count_) % K;
for (std::size_t i = 0; i < count_; ++i)
sum += data_[(oldest + i) % K].value;
return sum;
}
};

template <typename Log>
long long run(Log& log, long long n) {
for (long long i = 0; i < n; ++i)
log.add({i + 1});
return log.checksum();
}

int main() {
constexpr long long N = 2000000;
ShiftLog shifted;
CircularLog circular;

const auto s0 = std::chrono::steady_clock::now();
const long long sumShift = run(shifted, N);
const auto s1 = std::chrono::steady_clock::now();
const long long sumCircular = run(circular, N);
const auto s2 = std::chrono::steady_clock::now();

const auto shiftMs =
std::chrono::duration_cast<std::chrono::milliseconds>(s1-s0).count();
const auto circularMs =
std::chrono::duration_cast<std::chrono::milliseconds>(s2-s1).count();

std::cout << "Shift checksum: " << sumShift
<< "\nCircular checksum: " << sumCircular << '\n';
std::cout << "Shift ms: " << shiftMs
<< "\nCircular ms: " << circularMs << '\n';
}
```

The two checksums must match and should equal 19999955 for N=2000000. Run with the same compiler flags and machine conditions for both variants. Repeat several times and compare medians to reduce noise. If the measured time does not show a clear difference at capacity 10, explain that the small fixed capacity and timer resolution can hide a constant-factor improvement; the asymptotic insertion work is still different.

Measurement record – fill after your own execution

Build/run setting	Shift array	Circular array
Compiler and version		
Flags		
N / repetitions	2,000,000 / ____	2,000,000 / ____
Checksum	19,999,955 (expected)	19,999,955 (expected)
Elapsed time (ms)	_____	_____
Screenshot filename	_____	_____

F. Defend the final design (2 marks)

The circular buffer meets the measurable correctness objective if both checksums match and the last ten entries are identical and chronological. It also reduces full-buffer insertion from O(k) shifts to O(1) writes, at the cost of slightly more indexing logic and a more careful chronological traversal. A shift array may still be preferable for very small histories where simplicity matters more than insertion cost, or when the history must remain physically contiguous in oldest-to-newest order at all times. For this account statement, the circular design is recommended because it preserves fixed memory and makes every insertion constant time.

Q7 evidence is incomplete until you run the comparison: insert the real terminal screenshots and fill the measurement table. Do not submit blank timing fields as if they were results.

FINAL VALIDATION AND SUBMISSION CHECKLIST

Program verification status

Program	What was checked in this preparation	Your required evidence
Q1	C++17 syntax and logic; prediction: 1 12 13 / 1 100 / 16 13 6; fee styles	Your compile/run screenshot
Q2	Array maximum reference updates 95 to 97; dangling reference excluded from execution	Run screenshot and compiler warning screenshot for unsafe example
Q3	Six overload calls resolve to intended overloads; illustrative tariff calculations checked	Run screenshot and original ambiguity diagnostic
Q4	Circular log retains newest ten transactions in order; balance and statement checked	Run screenshot with more than ten transactions
Q5	Token IDs and BusTicket seat allocation checked	Run screenshot and two-call function test
Q6	205-minute expected result and noon/equal-time edge cases checked	Run screenshot and four benchmark measurements
Q7	Shift and circular implementations compile; identical expected checksum derived	Your comparative timings and screenshots

Validation note: the C++ source examples were compiled and run during preparation in a local validation environment using g++ 14.2.0, not g++ 13.3.0. Therefore the report does not claim that your stated compiler produced these results. Recompile every program with your own g++ 13.3.0 installation using the command below, capture the genuine outputs, and check that they match the predictions. No personal execution screenshots or personal benchmark timings have been fabricated.

Recommended compile command

```
g++ -std=c++17 -Wall -Wextra -Wpedantic Q1.cpp -o Q1
./Q1
```

Repeat the command for Q2.cpp through Q7.cpp, changing the filename and output executable. For benchmark work, add the required optimisation flags (-O0 or -O2) while keeping all other conditions the same.

README.md template

```
# DSA01 – Assignment 2 (CO2)

Student: Nirmal Kumar T
Register Number: 1972260020
Department: B.Tech CSE-AI
Academic Year: 2026–2030
Semester: 2
Faculty: Amirtha S
Compiler target: g++ 13.3.0
Language standard: C++17

Repository:
https://github.com/Nirmal637/DSA01-1972260020-Assignment2

## Compile
g++ -std=c++17 -Wall -Wextra -Wpedantic Q1.cpp -o Q1

## Run
./Q1

Repeat for Q2.cpp, Q3.cpp, Q4.cpp, Q5.cpp, Q6.cpp and Q7.cpp.

## Task results
- Q1: Compare call by value, reference and pointer.
- Q2: Return a reference to the maximum array element safely.
- Q3: Resolve overloads using distinct argument types and defaults.
- Q4: Store the latest ten transactions using a circular index.
- Q5: Allocate bus seats using static class data.
- Q6: Compare Time objects with friend functions.
- Q7: Compare shifting and circular-buffer transaction logs.

## Evidence
Add genuine terminal screenshots and benchmark records
for each question. Never add predicted output as an
execution screenshot.
```

GitHub folder structure

```
DSA01-1972260020-Assignment2/
|-- README.md
|-- Q1/
| |-- Q1.cpp
| |-- output.png
|-- Q2/
| |-- Q2.cpp
| |-- dangling_reference_warning.png
|-- Q3/
| |-- Q3.cpp
| |-- output.png
|-- Q4/
| |-- Q4.cpp
| |-- output.png
|-- Q5/
| |-- Q5.cpp
| |-- output.png
|-- Q6/
| |-- Q6.cpp
| |-- benchmark_O0_O2.png
|-- Q7/
|-- Q7.cpp
|-- comparison.png
```

Final submission checklist

- All seven questions and all supplied TODO sections are completed in the repository.
- Each question has its own folder and C++ source file.
- Every source file compiles under g++ 13.3.0 with -std=c++17 -Wall -Wextra.
- Predictions were written before execution; observed output is compared against them.
- The dangling-reference example is compiled for its warning only and is never dereferenced.
- Q6 inline/non-inline timing measurements are genuine and include both -O0 and -O2 runs.
- Q7 uses identical input counts for both implementations; checksums match; measured times are recorded honestly.
- README.md contains the exact commands, run steps and task summaries.
- At least one progressive commit exists for each question.
- The repository is accessible to faculty and the same link appears on the cover page.

Repository link: https://github.com/Nirmal637/DSA01-1972260020-Assignment2

Final reminder: The PDF is a complete prepared report and code reference. Before submission, replace the evidence blanks with your own screenshots and measured values, verify every program on g++ 13.3.0, and push all source files and evidence to the linked repository.